

Backend Basics

Phase 1 — Complete Study Guide

Shiwang Chaudhary · B.Tech CSE · NoteMandi Stack
Node.js · Express · MongoDB · Firebase Auth · Cloudinary

Topics Covered

- 01 · HTTP & HTTPS
- 02 · REST API Design
- 03 · Authentication & Authorization
- 04 · Databases — SQL vs NoSQL
- 05 · Database Performance
- 06 · Backend Architecture
- 07 · Validation & Error Handling
- 08 · Caching & Redis
- 09 · File Handling & Cloud Storage
- 10 · Security Basics

HTTP & HTTPS

The foundation of every web API you will ever build

HTTP (HyperText Transfer Protocol) is the language that browsers and servers use to talk to each other. Every time your Flutter app calls a backend API, it sends an HTTP request and receives an HTTP response. HTTPS is simply HTTP with TLS encryption on top — it protects data in transit.

Request & Response Cycle

Every HTTP interaction is a request-response pair. The client (Flutter app) sends a Request; the server (your Node.js/Express app) returns a Response.

Anatomy of an HTTP Request

- **Method** — what action to perform (GET, POST, PUT, PATCH, DELETE)
- **URL** — the resource address (e.g. /api/notes/123)
- **Headers** — metadata like Content-Type, Authorization
- **Body** — optional data payload (JSON for POST/PUT/PATCH)

Anatomy of an HTTP Response

- **Status Code** — 3-digit number indicating success or failure
- **Headers** — metadata like Content-Type, Set-Cookie
- **Body** — the actual data (JSON, HTML, file, etc.)

HTTP Methods

Method	Purpose	Has Body?	Idempotent?
GET	Read a resource	No	Yes
POST	Create a new resource	Yes	No
PUT	Replace entire resource	Yes	Yes
PATCH	Update part of a resource	Yes	Yes
DELETE	Remove a resource	No	Yes

■ *Idempotent = calling it multiple times gives the same result. GET /notes/1 always returns the same note; POST /notes creates a new note every time.*

Status Codes — The Ones You Must Know

Code	Name	When to use
200	OK	Successful GET / PUT / PATCH
201	Created	Successful POST (new resource created)
204	No Content	Successful DELETE (nothing to return)
400	Bad Request	Client sent invalid data (validation fail)

401	Unauthorized	Not logged in / token missing
403	Forbidden	Logged in but not allowed
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate resource (email already taken)
422	Unprocessable Entity	Semantically wrong (e.g. future birth date)
500	Internal Server Error	Something crashed on your server

Headers You Will Use Daily

Request Headers

- **Content-Type: application/json** — telling server you're sending JSON
- **Authorization: Bearer <token>** — sending JWT token
- **Accept: application/json** — telling server what format you want back

Response Headers

- **Content-Type: application/json** — server telling you the response is JSON
- **Set-Cookie** — server setting a cookie in your browser
- **Access-Control-Allow-Origin** — CORS header (covered in Security section)

Cookies vs Headers for Auth

There are two main ways to send authentication credentials with each request. Firebase Auth (which you use in NoteMandi) sends a token via the Authorization header. Session-based auth (like traditional web apps) uses cookies instead.

Method	Storage	Sent automatically?	Used in
JWT in Header	localStorage / memory	No — must add manually	SPAs, Mobile Apps, APIs
Session Cookie	Browser cookie store	Yes — browser adds it	Traditional web apps

HTTPS & TLS — Just Enough to Know

TLS (Transport Layer Security) encrypts the data between client and server so no one can read it in transit. When you deploy NoteMandi's backend to Railway or Render, HTTPS is handled for you automatically. You just need to understand why it matters — without HTTPS, a user's JWT token could be stolen mid-transit.

- HTTPS = HTTP + TLS encryption
- Protects against man-in-the-middle attacks
- Required for sending passwords, tokens, and sensitive data
- Your cloud provider (Railway, Render) gives you a free TLS certificate

■ *In NoteMandi: Firebase Auth already enforces HTTPS. When you build your own Node.js routes, ensure your deployment platform has HTTPS enabled (Railway and Render do this by default).*

REST API Design

How to structure your API routes the right way

REST (Representational State Transfer) is a set of rules for designing web APIs. A RESTful API uses HTTP methods and URLs to represent resources and actions. It's what makes your API predictable and easy to understand.

Resource Naming — The Golden Rules

- Use **nouns**, not verbs — /notes not /getNotes
- Use **plural** names — /notes not /note
- Use **kebab-case** for multi-word resources — /uploaded-files
- Use **nesting** for relationships — /users/:userId/notes
- Keep URLs **lowercase**

Good vs Bad URL Design

Bad ■	Good ■	Why
/getNotes	/notes	Use noun, not verb
/createNote	POST /notes	Method expresses action
/note/delete/5	DELETE /notes/5	Method + noun + ID
/userNotes/123	/users/123/notes	Nested resource
/Notes	/notes	Always lowercase

Standard CRUD Routes Pattern

This is the pattern you should follow for every resource in NoteMandi:

GET	/api/notes	→ Get all notes (with pagination)
GET	/api/notes/:id	→ Get a single note by ID
POST	/api/notes	→ Create a new note
PUT	/api/notes/:id	→ Replace a note completely
PATCH	/api/notes/:id	→ Update specific fields of a note
DELETE	/api/notes/:id	→ Delete a note

Idempotency

An operation is idempotent if doing it once or ten times gives the same result. This matters because networks can fail and clients may retry requests.

- **GET, PUT, DELETE** — idempotent (safe to retry)
- **POST** — NOT idempotent (creates a new resource each time)
- **PATCH** — usually idempotent (setting a value repeatedly is fine)

Pagination

Never return all records at once — it will crash your app when you have 10,000 notes. Use pagination to return data in pages.

Offset-Based Pagination (simplest)

```
GET /api/notes?page=2&limit;=10

// Express handler
const page = parseInt(req.query.page) || 1;
const limit = parseInt(req.query.limit) || 10;
const skip = (page - 1) * limit;

const notes = await Note.find().skip(skip).limit(limit);
const total = await Note.countDocuments();

res.json({
  data: notes,
  pagination: { page, limit, total, totalPages: Math.ceil(total/limit) }
});
```

Filtering & Sorting

```
GET /api/notes?subject=math&sort;=createdAt■=desc

// Express handler
const { subject, sort = 'createdAt', order = 'desc' } = req.query;
const filter = subject ? { subject } : {};
const sortObj = { [sort]: order === 'desc' ? -1 : 1 };
const notes = await Note.find(filter).sort(sortObj);
```

Response Structure — Be Consistent

Pick a response shape and stick to it across your entire API:

```
// Success response
{
  "success": true,
  "data": { ... },
  "message": "Note created successfully"
}

// Error response
{
  "success": false,
  "error": "Note not found",
  "statusCode": 404
}
```

■ *NoteMandi audit: Check your existing Express routes — do they all follow noun-based URLs? Do error responses have a consistent shape? Fixing these is a real interview talking point.*

Authentication & Authorization

Who are you? What are you allowed to do?

Authentication = verifying identity (logging in). Authorization = verifying permissions (can this user delete this note?). You already use Firebase Auth in NoteMandi — this section explains what's happening under the hood and adds JWT knowledge for when you build auth yourself.

Sessions vs JWT — The Key Difference

Aspect	Session-Based	JWT-Based
Storage	Server stores session in DB/memory	Server stores nothing — token is self-contained
Scalability	Hard to scale (shared session store)	Easy to scale (any server can verify)
Revocation	Easy — delete session from DB	Hard — need a blacklist or short expiry
Used in	Traditional web apps	APIs, mobile apps (NoteMandi)
Where stored	Cookie (HttpOnly)	localStorage, memory, or secure cookie

JWT — How It Actually Works

A JWT (JSON Web Token) is a string with three parts separated by dots: Header.Payload.Signature. The server signs it with a secret key. Anyone can read the payload, but only the server can create a valid signature.

JWT Structure

```
eyJhbGciOiJIUzI1NiJ9 ← Header (algorithm)
.eyJ1c2VySWQiOiIXMjMifQ ← Payload (your data)
.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c ← Signature
```

Creating and Verifying JWT in Node.js

```
const jwt = require('jsonwebtoken');

// Create token (after login)
const token = jwt.sign(
  { userId: user._id, role: user.role }, // payload
  process.env.JWT_SECRET,                // secret key
  { expiresIn: '7d' }                    // expiry
);

// Verify token (middleware)
const decoded = jwt.verify(token, process.env.JWT_SECRET);
// decoded = { userId: '...', role: 'student', iat: ..., exp: ... }
```

Access Tokens & Refresh Tokens

Short-lived access tokens are more secure, but users would have to log in frequently. Refresh tokens solve this — they're long-lived tokens used only to get a new access token.

Token	Expiry	Purpose	Stored where
Access Token	15 min – 1 hour	Auth every request	Memory / localStorage
Refresh Token	7 – 30 days	Get a new access token	HttpOnly Cookie (more secure)

Firestore Auth — What's Happening Internally

When you use Firestore Auth in NoteMandi, Firestore handles the entire auth flow. Your Flutter app receives an ID token (a JWT). You send this token to your Node.js backend. Your backend verifies it using the Firestore Admin SDK.

```
// Backend: verify Firestore ID token
const admin = require('firebase-admin');

async function verifyFirestoreToken(req, res, next) {
  const token = req.headers.authorization?.split('Bearer ')[1];
  if (!token) return res.status(401).json({ error: 'No token' });

  try {
    const decoded = await admin.auth().verifyIdToken(token);
    req.user = decoded; // { uid, email, ... }
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
}
```

Password Hashing — For When You Build Your Own Auth

Never store plain-text passwords. Always hash them using bcrypt. Bcrypt adds a random 'salt' before hashing, so two identical passwords produce different hashes.

```
const bcrypt = require('bcrypt');

// Hash password (on register)
const saltRounds = 12;
const hash = await bcrypt.hash(plainPassword, saltRounds);
// Store 'hash' in database, never 'plainPassword'

// Verify password (on login)
const isMatch = await bcrypt.compare(plainPassword, storedHash);
// isMatch = true or false
```

Role-Based Access Control (RBAC)

In NoteMandi, you likely have students (buyers) and sellers (uploaders). RBAC lets you restrict routes based on a user's role.

```
// Middleware: require specific role
function requireRole(role) {
  return (req, res, next) => {
    if (req.user.role !== role) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
}
```

```
    }
    next();
  };
}
// Usage in routes
router.post('/notes', verifyToken, requireRole('seller'), uploadNote);
router.get('/notes', verifyToken, getNotes); // all roles
```

■ Interview question: 'What's the difference between 401 and 403?' → 401 = not authenticated (no valid token). 403 = authenticated but not authorized (wrong role).

Databases — SQL vs NoSQL

When to use MongoDB and when to use PostgreSQL

You currently use MongoDB for NoteMandi, which is a NoSQL database. For FAANG interviews, you need to understand both SQL (PostgreSQL) and NoSQL (MongoDB) — when to use each, their trade-offs, and basic schema design for both.

SQL vs NoSQL — The Real Difference

Aspect	SQL (PostgreSQL)	NoSQL (MongoDB)
Structure	Fixed schema (tables, columns)	Flexible schema (documents, JSON)
Relations	JOINS between tables	Embed or reference documents
Consistency	Strong (ACID transactions)	Eventual by default (configurable)
Scaling	Vertical (bigger machine)	Horizontal (more machines)
Query	SQL language	MongoDB query API / Aggregation
Best for	Financial, relational data	Content, catalogs, flexible schemas

MongoDB Schema Design

Two key patterns: Embedding (put related data inside one document) vs Referencing (store an ID and look it up separately).

When to Embed

- Data is always accessed together (user profile + address)
- One-to-few relationships (post → 5 comments)
- Child data doesn't need to exist independently

```
// Embedded – note with tags inside
{
  "_id": "note123",
  "title": "Thermodynamics Notes",
  "subject": "Physics",
  "tags": ["heat", "entropy", "laws"], // embedded array
  "uploadedBy": "userId456"
}
```

When to Reference

- Data is accessed independently (users, orders)
- One-to-many or many-to-many relationships
- Related document can be large

```
// Referenced – purchase references user and note
{
  "_id": "purchase789",
```

```
"userId": "userId456",    // reference to User collection
"noteId": "note123",      // reference to Note collection
"purchasedAt": "2025-01-15T10:30:00Z",
"amount": 49
}
```

SQL Basics — What You Need for Interviews

Even though NoteMandi uses MongoDB, interview questions often test SQL. Know these core concepts.

Core SQL Concepts

- **Primary Key** — unique identifier for each row (like MongoDB's `_id`)
- **Foreign Key** — a column that references a primary key in another table
- **Normalization** — reducing data duplication by splitting into tables
- **JOIN** — combining rows from two tables based on a related column

Basic SQL Queries

```
-- Select with filter
SELECT * FROM notes WHERE subject = 'Physics';

-- Join: get note with uploader's name
SELECT notes.title, users.name
FROM notes
JOIN users ON notes.user_id = users.id
WHERE notes.id = 1;

-- Aggregate: count notes per subject
SELECT subject, COUNT(*) as total
FROM notes
GROUP BY subject
ORDER BY total DESC;
```

■ *NoteMandi parallel: Think of your MongoDB collections as SQL tables. `users` → `users table`, `notes` → `notes table`, `purchases` → `purchases table` with foreign keys to both.*

Database Performance

Indexes, query optimization, and ACID transactions

Indexes — The Most Important Performance Tool

Without an index, MongoDB or PostgreSQL must scan every single document/row to find your data. An index is like a book's index — it lets the database jump directly to the right location.

When to Create an Index

- Any field you query frequently (subject, userId, createdAt)
- Any field you sort by regularly
- Foreign key fields (noted in purchases collection)
- Fields used in unique constraints (email)

```
// MongoDB – create indexes
db.notes.createIndex({ subject: 1 }); // single field
db.notes.createIndex({ userId: 1, createdAt: -1 }); // compound
db.users.createIndex({ email: 1 }, { unique: true }); // unique

// Mongoose model definition with index
const noteSchema = new Schema({
  subject: { type: String, index: true },
  userId: { type: ObjectId, ref: 'User', index: true },
  createdAt: { type: Date, default: Date.now },
});
```

Check if Your Query Uses an Index

```
// MongoDB: explain() shows query execution plan
db.notes.find({ subject: 'Physics' }).explain('executionStats')

// Look for: 'IXSCAN' (index scan) = good
//           'COLLSCAN' (collection scan) = needs index
```

ACID Transactions

ACID ensures database operations are reliable, especially when multiple things happen together. Example: a student buys a note — you must deduct payment AND record the purchase atomically.

Property	Meaning	Example
Atomicity	All or nothing	Payment deducted AND purchase recorded, or neither
Consistency	DB stays in valid state	Can't have a purchase without a valid noteld
Isolation	Concurrent ops don't conflict	Two users buying simultaneously don't corrupt data
Durability	Committed data persists	Confirmed purchase survives a server crash

```
// MongoDB transaction example
const session = await mongoose.startSession();
```

```
session.startTransaction();
try {
  await Purchase.create([{ userId, noteId, amount }], { session });
  await User.updateOne({ _id: userId },
    { $inc: { walletBalance: -amount } }, { session });
  await session.commitTransaction();
} catch (err) {
  await session.abortTransaction(); // rolls back everything
  throw err;
} finally {
  session.endSession();
}
```

■ *MongoDB transactions require a replica set. MongoDB Atlas (free tier) supports this by default — so NoteMandi can use transactions without extra setup.*

Backend Architecture

MVC, layered architecture, and clean code structure

Architecture is how you organize your code. Good architecture means: easy to debug, easy to test, easy to extend. The most common pattern for Node.js/Express is a Layered Architecture (a refined version of MVC).

Layered Architecture — The Pattern to Use

Layer	Responsibility	Files
Routes	Define URL endpoints, validate HTTP	routes/noteRoutes.js
Controllers	Handle req/res, call service layer	controllers/noteController.js
Services	Business logic — the rules of your app	services/noteService.js
Repository	Database queries only	repositories/noteRepo.js
Models	Database schema definition	models/Note.js

Why This Separation Matters

- If you switch from MongoDB to PostgreSQL, you only change the Repository layer
- Business logic (service) can be unit tested without hitting the database
- Controllers stay thin — they just parse requests and call services

Code Example — Note Upload Flow

```
// routes/noteRoutes.js
router.post('/', verifyToken, upload.single('file'), noteController.create);

// controllers/noteController.js
async create(req, res) {
  const result = await noteService.createNote(req.user.uid, req.body, req.file);
  res.status(201).json({ success: true, data: result });
}

// services/noteService.js
async createNote(userId, noteData, file) {
  const fileUrl = await cloudinaryService.upload(file);
  return noteRepo.create({ ...noteData, userId, fileUrl });
}

// repositories/noteRepo.js
async create(data) {
  return Note.create(data); // pure DB operation
}
```

■ Look at your current NoteMandi backend — if your route handlers directly call `Note.find()` or `Note.create()`, you're mixing layers. Refactoring to this pattern is a great interview talking point.

Validation & Error Handling

Never trust user input. Always handle errors gracefully.

Input Validation with Zod (recommended) or Joi

Validate every field coming in from the client before it touches your database. Zod is the modern choice — it works with TypeScript and gives excellent error messages.

```
const { z } = require('zod');

const createNoteSchema = z.object({
  title:    z.string().min(3).max(100),
  subject:  z.string().min(2),
  price:    z.number().min(0).max(999),
  semester: z.number().int().min(1).max(8),
});

// Middleware
function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return res.status(400).json({ error: result.error.flatten() });
    }
    req.validated = result.data;
    next();
  };
}

// In route
router.post('/', verifyToken, validate(createNoteSchema), noteController.create);
```

Centralized Error Handling

Instead of handling errors in every controller, use Express's error middleware. Any error thrown anywhere in your app funnels into one place.

```
// Custom error class
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

// In a service – throw errors cleanly
async getNoteById(id) {
  const note = await noteRepo.findById(id);
  if (!note) throw new AppError('Note not found', 404);
  return note;
}
```

```
// app.js – global error handler (must be last middleware)
app.use((err, req, res, next) => {
  const statusCode = err.statusCode || 500;
  res.status(statusCode).json({
    success: false,
    error: err.message || 'Internal Server Error',
  });
});
```

Logging with Winston

```
const winston = require('winston');
const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
  ],
});

// Use in services
logger.info('Note created', { noteId: note._id, userId });
logger.error('DB error', { error: err.message });
```

Caching & Redis

Make your app fast by not hitting the database every time

Caching stores frequently-accessed data in fast memory (RAM) so you don't query the database on every request. Redis is the industry standard cache.

Why Caching Exists

- Database queries are slow (~10-50ms); Redis reads are fast (~0.1ms)
- Reduce database load — the same 100 users browsing top notes shouldn't each trigger a DB query
- Scale without scaling your database

Cache-Aside Pattern (most common)

Cache-aside is the standard pattern: check cache first, if not found (cache miss) → hit DB, then store in cache.

```
const redis = require('ioredis');
const client = new redis(process.env.REDIS_URL);

async function getTopNotes() {
  const cacheKey = 'notes:top';

  // 1. Check cache
  const cached = await client.get(cacheKey);
  if (cached) return JSON.parse(cached); // cache HIT

  // 2. Cache MISS — query database
  const notes = await Note.find().sort({ downloads: -1 }).limit(20);

  // 3. Store in cache with 5 min TTL
  await client.setex(cacheKey, 300, JSON.stringify(notes));

  return notes;
}
```

Cache Invalidation

The hardest problem in caching: when your data changes, you must remove the stale cache entry.

```
// When a new note is uploaded, invalidate the top notes cache
async function createNote(data) {
  const note = await Note.create(data);
  await client.del('notes:top'); // invalidate cache
  return note;
}
```

■ *NoteMandi use case: Cache the 'top notes' and 'notes by subject' listings. These are read far more than written, making them perfect cache candidates.*

File Handling & Cloud Storage

You already use Cloudinary — now understand why each piece exists

You're already using Cloudinary in P-STOR and NoteMandi. This section explains the full flow and fills in the 'why' behind each step.

The File Upload Flow

- Client selects a file (Flutter picks a PDF or image)
- Client sends a multipart/form-data POST request to your backend
- Backend receives the file via multer middleware
- Backend uploads to Cloudinary (or S3)
- Cloud storage returns a URL
- Backend saves the URL to MongoDB
- Backend returns success with the file URL

Multer — Handling Uploads in Express

```
const multer = require('multer');
// Store file in memory (then send to Cloudinary)
const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 10 * 1024 * 1024 }, // 10MB limit
  fileFilter: (req, file, cb) => {
    const allowed = ['application/pdf', 'image/jpeg', 'image/png'];
    if (!allowed.includes(file.mimetype)) {
      return cb(new AppError('Only PDF and images allowed', 400));
    }
    cb(null, true);
  }
});
router.post('/notes', verifyToken, upload.single('file'), noteController.create);
```

Cloudinary Upload

```
const cloudinary = require('cloudinary').v2;
async function uploadToCloudinary(fileBuffer, folder) {
  return new Promise((resolve, reject) => {
    const stream = cloudinary.uploader.upload_stream(
      { folder, resource_type: 'auto' },
      (error, result) => {
        if (error) reject(error);
        else resolve(result.secure_url);
      }
    );
    stream.end(fileBuffer);
  });
}
```

```
});  
}
```

Signed URLs

A signed URL is a time-limited, access-controlled URL for private files. In NoteMandi, a paid note PDF should only be accessible to users who purchased it — a signed URL expires after 15 minutes so it can't be shared.

```
// Cloudinary signed URL (expires in 15 minutes)  
const signedUrl = cloudinary.url('notes/thermodynamics.pdf', {  
  sign_url: true,  
  expires_at: Math.floor(Date.now() / 1000) + 900, // 15 min  
  resource_type: 'raw',  
});
```

Security Basics

The attacks you must understand and prevent

CORS — Cross-Origin Resource Sharing

Browsers block requests from one domain to another by default (same-origin policy). CORS headers tell the browser which origins are allowed. Your Flutter web app on localhost:3000 calling your API on localhost:5000 will fail without CORS.

```
const cors = require('cors');
app.use(cors({
  origin: ['https://notemandi.app', 'http://localhost:3000'],
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));
```

■■ *Never use origin: '*' in production — it allows ANY website to call your API.*

Common Attacks You Must Know

Attack	What it is	Prevention
SQL Injection	Malicious SQL in input field	Use parameterized queries / ORMs (Mongoose, Prisma)
XSS	Injecting scripts into web pages	Sanitize input, use Content-Security-Policy header
CSRF	Tricking user into making request	CSRF tokens, SameSite cookies, check Origin header
Brute Force	Trying passwords repeatedly	Rate limiting, account lockout

Rate Limiting

Limit how many requests a client can make in a time window. Prevents brute force attacks and API abuse.

```
const rateLimit = require('express-rate-limit');
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // 100 requests per window
  message: { error: 'Too many requests, slow down.' }
});
// Stricter limit for auth routes
const authLimiter = rateLimit({ windowMs: 60000, max: 5 });
app.use('/api/', limiter);
app.use('/api/auth/', authLimiter);
```

Environment Variables & Secrets

Never hardcode secrets. Always use environment variables.

```
# .env file (never commit this to git)
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/notemandi
JWT_SECRET=your-256-bit-secret-key
CLOUDINARY_API_SECRET=abc123...
FIREBASE_PRIVATE_KEY=...

# In Node.js
require('dotenv').config();
const uri = process.env.MONGODB_URI;

# .gitignore
.env
```

■■ Add *.env* to *.gitignore* *BEFORE* your first commit. GitHub will alert you if you accidentally push secrets, but the damage is already done.

Quick Reference Cheatsheet

Topic	The 10-Second Summary
HTTP Methods	GET=read, POST=create, PUT=replace, PATCH=update, DELETE=remove
Status Codes	2xx=success, 4xx=client error, 5xx=server error
REST URLs	Nouns not verbs, plural, nested for relations: /users/:id/notes
JWT	Header.Payload.Signature — server signs, client stores, server verifies
401 vs 403	401=not logged in, 403=logged in but not allowed
MongoDB embed vs ref	Embed if always accessed together; reference if accessed independently
Indexes	Create on fields you filter/sort by frequently; use explain() to verify
ACID	All-or-nothing, valid state, isolated, durable — use transactions for multi-step ops
Layers	Route → Controller → Service → Repository → Model
Cache-aside	Check cache → miss → query DB → store in cache with TTL
CORS	Whitelist specific origins in production, never use *
Rate Limiting	Max 100 req/15min general, max 5 req/min for auth routes
Env variables	.env file + dotenv + .gitignore — never hardcode secrets
Signed URLs	Time-limited URLs for private files — use for paid note PDFs in NoteMandi

Build → Break → Fix → Understand

Study each concept then immediately apply it to NoteMandi.